Trabajo Práctico N.º 3

Base de datos relacional de reseñas de vinos

Diseño, implementación en MySQL y análisis multidimensional

Conjunto de datos: winemag-data-130k-v2 (WineEnthusiast)

Autor: Lucas Llano

Fecha: 14 de octubre de 2026

Índice

1. Introducción y datos 2

1.1 Objetivo 2

1.2 El archivo y su limpieza 2

2. Modelo relacional 2

2.1 Entidades 2

2.2 Relaciones 3

2.3 Decisiones de diseño 4

3. Implementación en MySQL 5

3.1 Tablas y restricciones 5

3.2 Carga 6

3.3 Pruebas de integridad y transacciones 6

4. Consultas SQL 7

4.1 Vinos por país (C01) 7

4.2 Variedades más comunes en una región (C02) 8

4.3 Precio promedio por puntuación (C04) 9

4.4 Subconsulta correlacionada (C06) 10

4.5 Cuartiles de precio (C10) 11

4.6 Otras consultas del script 12

5. Análisis multidimensional 12

5.1 Roll-up: país y rango de precio (M01) 12

5.2 Pivot: país y categoría de puntaje (M03) 13

5.3 Dice: países, variedades y precio (M05) 14

5.4 Matriz país y rango de precio (M07) 15

5.5 Drill-down, slice y cubo 17

6. Desafíos 17

7. Conclusiones y trabajo futuro 17

Referencias 18

1. Introducción y datos

1.1 Objetivo

Para este trabajo armé una base de datos relacional con las reseñas de vinos de WineEnthusiast (winemag-data-130k-v2.csv). El archivo original es una tabla plana de casi 130.000 filas donde cada reseña repite el país, la provincia, la bodega, la variedad y el catador, así que la idea fue normalizarlo, cargarlo en MySQL con sus restricciones y después usarlo para responder preguntas con SQL y hacer el análisis multidimensional que pide la consigna.

Usé MySQL 8 para la base y Python para limpiar el archivo y hacer los gráficos. Todo el SQL está en TP3_Vinos_Lucas_Llano.sql, que crea la base desde cero y se corre de una vez.

1.2 El archivo y su limpieza

El "Documento extra" que nos pasaron no era el CSV sino un .xlsm con una sola columna, donde cada celda tiene una línea entera del CSV con comas y comillas. Tenía dos problemas. Unas 430 filas estaban partidas en varias columnas, porque el archivo se abrió con punto y coma como separador y cada descripción que tenía un ";" quedó cortada ahí. Además los acentos y algunos símbolos estaban mal codificados, y aparecían cosas como "VulkÃ Bianco" o "Oâ€™Keefe".

Lo arreglé con un script corto de Python antes de pasar a MySQL y quedaron 129.971 reseñas con sus 14 campos, igual que el dataset original.

Antes de diseñar el modelo miré qué tan completos estaban los datos. region_2 falta en el 61% de las filas (solo se usa para vinos de Estados Unidos), el precio falta en el 6,9% y una de cada cinco reseñas no tiene catador, así que esa relación tenía que ser opcional. También encontré 9.983 filas duplicadas, idénticas en todo salvo el índice, que saqué durante la carga.

2. Modelo relacional

2.1 Entidades

Empecé viendo qué datos se repetían en el archivo: cada provincia siempre va con el mismo país, cada región con la misma provincia y cada catador con su usuario de Twitter. Separando eso en tablas propias el modelo queda en tercera forma normal.

Lo que más me costó fue definir qué es un "vino". Mi primera idea fue usar el título de la reseña como identificador, pero al agrupar por título aparecieron 578 vinos con precios distintos según la reseña y 751 con puntajes distintos. O sea que el precio y los puntos son

de cada evaluación, no del vino, y por eso quedaron en resena. El vino terminó siendo la etiqueta: título, designación, bodega, variedad y origen.

El modelo quedó en nueve tablas: pais, provincia y region forman la jerarquía geográfica, zona guarda las agrupaciones de region_2, bodega, variedad y catador son catálogos, vino es la etiqueta y resena la evaluación (que después es la tabla de hechos del análisis multidimensional).

Tabla	Clave primaria	Claves foráneas	Otras restricciones	Filas
pais	id_pais		nombre UNIQUE	43
provincia	id_provincia	id_pais → pais	UNIQUE (id_pais, nombre)	425
region	id_region	id_provincia → provincia	UNIQUE (id_provincia, nombre); UNIQUE (id_region, id_provincia)	1.231
zona	id_zona		nombre UNIQUE	17
bodega	id_bodega		nombre UNIQUE	16.728
variedad	id_variedad		nombre UNIQUE	707
catador	id_catador		nombre UNIQUE; CHECK usuario_twitter LIKE '@%'	19
vino	id_vino	id_bodega, id_variedad, id_provincia, (id_region, id_provincia), id_zona	clave_natural UNIQUE; CHECK anio_cosecha 1900-2017	119.082
resena	id_resena	id_vino → vino; id_catador → catador	CHECK puntos 80-100; CHECK precio > 0; trigger de validación	119.988

Tabla 1. Esquema de las tablas y filas cargadas.

Además de las claves, vino guarda titulo, designacion y anio_cosecha (lo saco del título), y resena guarda puntos, precio, descripcion y la fecha de carga.

2.2 Relaciones

Todas las relaciones son de uno a muchos. No necesité tablas intermedias porque en el dataset cada vino tiene una sola variedad y cada reseña un solo catador.

Relación	Cardinalidad	Comentario
pais - provincia	1 : 1..N	Cada provincia pertenece a un país.
provincia - region	1 : 0..N	Hay provincias sin regiones cargadas.
provincia - vino	0..1 : 0..N	59 reseñas no tienen origen.
region - vino	0..1 : 0..N	La región falta en el 16% de las filas.
zona - vino	0..1 : 1..N	Solo los vinos de EE. UU. tienen zona.
bodega - vino	1 : 1..N	Todo vino tiene bodega.
variedad - vino	0..1 : 1..N	Una sola reseña no tiene variedad.
vino - resena	1 : 1..N	694 vinos tienen más de una reseña.
catador - resena	0..1 : 1..N	El 20,8% de las reseñas no tiene catador.

Tabla 2. Relaciones y cardinalidades.

2.3 Decisiones de diseño

Usé claves subrogadas (AUTO_INCREMENT) en todas las tablas menos resena, donde dejé el índice del CSV para poder rastrear cada fila hasta el archivo. Los nombres no servían de clave: son largos y además se repiten, por ejemplo hay dos regiones "Niagara Escarpment", una en Ontario y otra en Nueva York. Por eso el nombre de la región es único solo dentro de su provincia.

vino tiene referencia a la provincia y a la región, y como la región ya implica la provincia, podía pasar que un vino quedara con una región de Mendoza y una provincia de California. Para evitarlo hice la FK hacia region compuesta, (id_region, id_provincia), contra una clave alternativa de region. Así MySQL rechaza la combinación (lo probé en la sección 3.3).

Para no cargar dos veces el mismo vino puse una columna clave_natural que combina el título, la designación y las FK, con un UNIQUE encima. No lo hice directamente sobre esas columnas porque varias pueden ser NULL y MySQL deja repetir NULL en un UNIQUE.

Otras decisiones más chicas: usuario_twitter no es UNIQUE porque Jeff Jenssen y Mike DeSimone comparten @worldwineguys; las FK no tienen CASCADE, para que borrar un país por error no se lleve miles de reseñas; y configuré la base para que distinga acentos pero no mayúsculas, así "Château" y "Chateau" son bodegas distintas pero "Château La Barotte" y "Château la Barotte" no. Con eso las 16.755 grafías de bodegas del CSV quedaron en 16.728.

El diagrama está en notación pata de gallo, con PK subrayadas, FK marcadas y la cardinalidad en cada extremo. También va aparte en Diagrama_BD_Lucas_Llano.pdf.

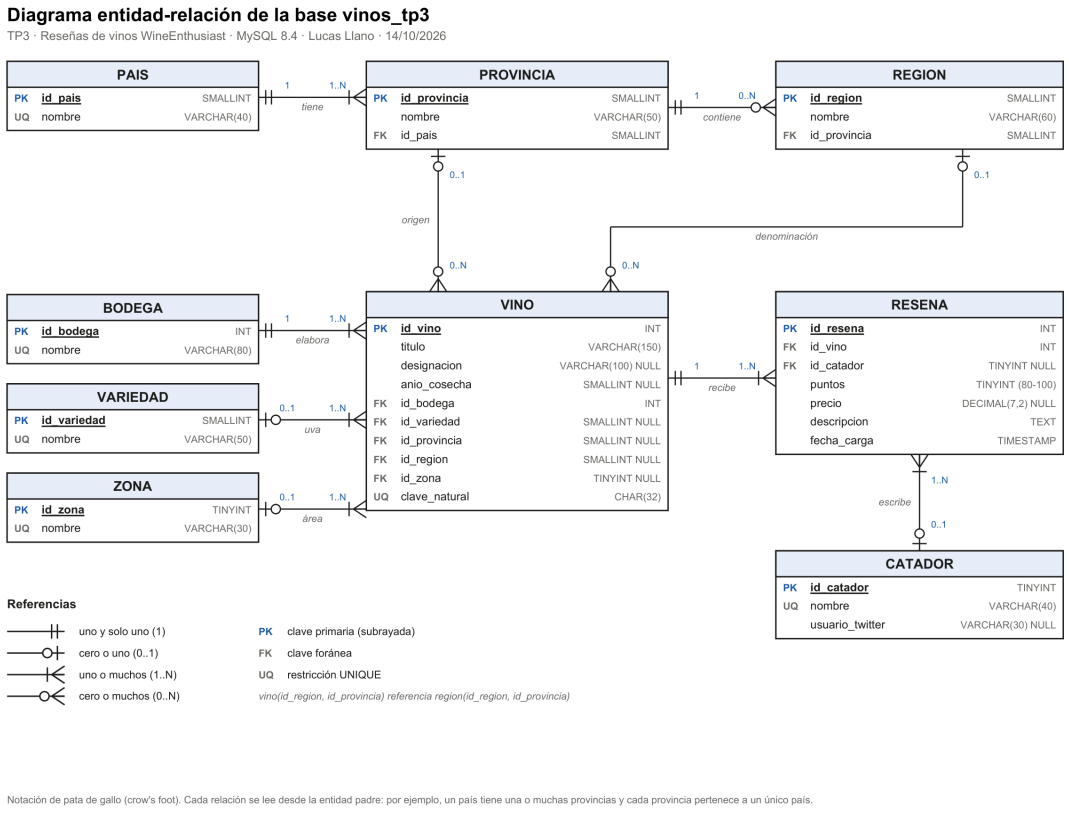


Figura 1. Diagrama entidad-relación de la base vinos_tp3.

3. Implementación en MySQL

3.1 Tablas y restricciones

Todas las tablas son InnoDB y le puse nombre a cada restricción (pk_, fk_, uq_, ck_) para que los errores se entiendan. Esta es resena, que es la que más reglas tiene:

```
CREATE TABLE resena (
    id_resena INT UNSIGNED NOT NULL, -- índice del CSV original
    id_vino INT UNSIGNED NOT NULL,
    id_catador TINYINT UNSIGNED NULL,
    puntos TINYINT UNSIGNED NOT NULL,
    precio DECIMAL(7,2) NULL, -- en dólares
    descripcion TEXT NOT NULL,
    fecha_carga TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_resena PRIMARY KEY (id_resena),
    CONSTRAINT ck_resena_puntos CHECK (puntos BETWEEN 80 AND 100),
    CONSTRAINT ck_resena_precio CHECK (precio IS NULL OR precio > 0),
    CONSTRAINT fk_resena_vino FOREIGN KEY (id_vino) REFERENCES vino (id_vino),
    CONSTRAINT fk_resena_catador FOREIGN KEY (id_catador) REFERENCES catador (id_catador),
    INDEX ix_resena_puntos (puntos),
    INDEX ix_resena_precio (precio)
) ENGINE = InnoDB;
```

El CHECK de puntos sale de la fuente: WineEnthusiast solo publica reseñas de 80 puntos o más. Los índices en puntos y precio los agregué porque casi todas las consultas filtran o agrupan por ahí.

3.2 Carga

El CSV no va directo a las tablas finales. Primero lo cargo en stg_resena, una tabla de staging sin restricciones, con LOAD DATA LOCAL INFILE, y ahí mismo paso los vacíos a NULL:

```
LOAD DATA LOCAL INFILE 'datos/winemag_limpio.csv'
INTO TABLE stg_resena
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"' ESCAPED BY ''
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id_original, @pais, @descripcion, @designacion, puntos, @precio, ...)
SET pais = NULLIF(@pais, ''), precio = NULLIF(@precio, ''), ...;
```

Separar la carga en dos pasos me sirvió para saber dónde estaba cada error: si fallaba LOAD DATA era el formato del archivo, si fallaba después era alguna regla del modelo.

La normalización se hace dentro de una transacción (START TRANSACTION ... COMMIT), así que si un INSERT viola algo el script se corta y no queda nada a medio cargar. Para los duplicados comparo todas las columnas de cada fila y me quedo con la primera de cada grupo. Después cargo las dimensiones de arriba hacia abajo con INSERT ... SELECT DISTINCT, resuelvo las claves en el staging y por último inserto vinos y reseñas. El año de cosecha lo saco del título; quedaron 3.613 vinos sin año, todos "NV" (sin cosecha declarada, típico de espumantes y fortificados).

Para controlar la carga comparé el staging con la tabla final: 129.971 filas menos 9.983 duplicados dan exactamente las 119.988 reseñas cargadas, y no se perdió ninguna.

También agregué algunos objetos que uso después: las funciones fn_rango_precio y fn_categoria_puntaje (esta sigue la escala de WineEnthusiast: Aceptable 80-82, Bueno 83-86, Muy bueno 87-89, Excelente 90-93, Soberbio 94-97, Clásico 98-100), el trigger tg_resena_validar que rechaza descripciones de menos de diez caracteres, la vista v_hechos_resena que junta la tabla de hechos con todas sus dimensiones, y el procedimiento sp_variedades_region.

3.3 Pruebas de integridad y transacciones

Para comprobar que las restricciones funcionan, el script intenta romperlas a propósito. Para que el script no se corte en cada error, las pruebas pasan por un procedimiento (sp_probar) que atrapa el error y muestra el mensaje del servidor.

Prueba	Respuesta del servidor
Puntaje fuera de escala (120)	Check constraint 'ck_resena_puntos' is violated.
Precio negativo	Check constraint 'ck_resena_precio' is violated.
FK a un vino inexistente	Cannot add or update a child row: a foreign key constraint fails (fk_resena_vino)
Clave primaria repetida	Duplicate entry '0' for key 'resena.PRIMARY'
País duplicado (UNIQUE)	Duplicate entry 'Argentina' for key 'pais.uq_pais_nombre'
Borrar un país con vinos (RESTRICT)	Cannot delete or update a parent row: a foreign key constraint fails (fk_provincia_pais)
Descripción vacía (trigger)	La descripción de la reseña es demasiado corta (mínimo 10 caracteres).
Región de otra provincia (FK compuesta)	Cannot add or update a child row: a foreign key constraint fails (fk_vino_region)

Tabla 3. Pruebas de integridad (los mensajes de FK están abreviados).

Las ocho fueron rechazadas. La última es la que más me interesaba: un vino con provincia Mendoza y región Napa Valley. Las dos claves existen, así que con dos FK simples habría pasado; la compuesta es la que lo frena.

Por último simulé una operación de mantenimiento para probar las transacciones:

```
START TRANSACTION;
UPDATE resena r JOIN vino v ... SET r.precio = r.precio * 1.10 WHERE pa.nombre = 'Argentina' ...;
SAVEPOINT antes_de_borrar;
DELETE r FROM resena r JOIN catador c ... WHERE c.nombre = 'Michael Schachner' AND r.puntos < 82;
ROLLBACK TO SAVEPOINT antes_de_borrar; -- vuelve el borrado
ROLLBACK; -- descarta también el aumento
```

El precio medio de Argentina pasó de 24,58 a 27,03 dentro de la transacción y volvió a 24,58 después del ROLLBACK. El DELETE había borrado 483 filas, que volvieron con el ROLLBACK TO SAVEPOINT.

4. Consultas SQL

En el script hay 14 consultas (C01 a C14). Acá muestro las que me parecieron más representativas, con algunos resultados recortados. Todas trabajan sobre las 119.988 reseñas sin duplicados (promedio general: 88,44 puntos y 35,62 dólares).

4.1 Vinos por país (C01)

Junta reseñas, vinos, provincias y países, y para el porcentaje usé una función de ventana sobre el total.

```
SELECT pa.nombre AS pais,
       COUNT(DISTINCT v.id_vino) AS vinos,
       COUNT(*) AS resenas,
       ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_resenas,
       ROUND(AVG(r.puntos), 2) AS puntaje_promedio
FROM resena r
JOIN vino v ON v.id_vino = r.id_vino
JOIN provincia p ON p.id_provincia = v.id_provincia
JOIN pais pa ON pa.id_pais = p.id_pais
GROUP BY pa.nombre
ORDER BY resenas DESC
LIMIT 15;
```

pais	vinos	resenas	pct_resenas	puntaje_promedio
US	50231	50457	42,07	88,56
France	19967	20353	16,97	88,87
Italy	17811	17940	14,96	88,58
Spain	6025	6116	5,10	87,24
Portugal	5222	5256	4,38	88,26
Chile	4179	4184	3,49	86,46
Argentina	3543	3544	2,96	86,65
Austria	3024	3034	2,53	90,13
Australia	2183	2197	1,83	88,56
Germany	1990	1992	1,66	89,90

Resultado de C01 (primeras 10 de 15 filas).

Estados Unidos tiene el 42,07% de las reseñas y junto con Francia e Italia llega casi al 74%. Es lógico porque la revista es estadounidense, pero hay que tenerlo en cuenta: un promedio "global" es casi un promedio de esos tres países. Vinos y reseñas dan casi lo mismo en todos los países porque la mayoría de las etiquetas se reseñó una sola vez. En puntaje, de los países de la tabla arriba están Austria (90,13) y Alemania (89,90) y abajo Chile (86,46) y Argentina (86,65).

4.2 Variedades más comunes en una región (C02)

```
SELECT va.nombre AS variedad,
       COUNT(*) AS resenas,
       ROUND(AVG(r.puntos), 2) AS puntaje_promedio,
       ROUND(AVG(r.precio), 2) AS precio_promedio
FROM resena r
JOIN vino v ON v.id_vino = r.id_vino
JOIN provincia p ON p.id_provincia = v.id_provincia
JOIN variedad va ON va.id_variedad = v.id_variedad
WHERE p.nombre = 'Mendoza Province'
GROUP BY va.nombre
ORDER BY resenas DESC
LIMIT 10;
```

variedad	resenas	puntaje_promedio	precio_promedio
Malbec	1265	87,55	28,54
Cabernet Sauvignon	456	86,25	20,19
Chardonnay	253	84,89	17,67
Red Blend	190	88,49	33,38
Torrontés	109	84,56	12,35
Bonarda	92	86,49	16,52
Bordeaux-style Red Blend	78	89,97	42,16
Sauvignon Blanc	58	84,14	13,72

Resultado de C02 (primeras 8 de 10 filas).

El Malbec tiene 1.265 de las 3.038 reseñas de Mendoza (41,6%) y puntúa mejor que el Cabernet Sauvignon y el Chardonnay de la provincia, aunque los mejor puntuados son los

cortes bordeleses (89,97), que también son más caros. El Torrontés es el más barato (12,35) y de los que menos puntúan. La misma pregunta para cualquier otra región se puede hacer con el procedimiento sp_variedades_region (C03 lo usa para Napa Valley).

4.3 Precio promedio por puntuación (C04)

```
SELECT r.puntos,
       COUNT(r.precio) AS resenas_con_precio,
       ROUND(AVG(r.precio), 2) AS precio_promedio,
       MIN(r.precio) AS precio_minimo,
       MAX(r.precio) AS precio_maximo
FROM resena r
GROUP BY r.puntos
ORDER BY r.puntos;
```

puntos	resenas_con_precio	precio_promedio	precio_minimo	precio_maximo
80	395	16,37	5,00	69,00
81	677	17,14	5,00	130,00
82	1750	18,90	4,00	150,00
83	2830	18,24	4,00	225,00
84	5860	19,25	4,00	225,00
85	8470	20,02	4,00	320,00
86	10748	22,12	4,00	170,00
87	14225	24,92	5,00	800,00
88	14209	28,80	6,00	3300,00
89	10307	31,99	7,00	500,00
90	12920	36,99	8,00	510,00
91	9806	43,37	7,00	2013,00
92	8244	51,29	11,00	750,00
93	5634	63,22	12,00	770,00
94	3328	81,52	13,00	1125,00
95	1387	109,34	20,00	973,00
96	481	159,47	20,00	2500,00
97	206	207,78	35,00	2000,00
98	69	245,49	50,00	1900,00
99	28	284,21	44,00	850,00
100	19	485,95	80,00	1500,00

Resultado de C04 (21 filas).

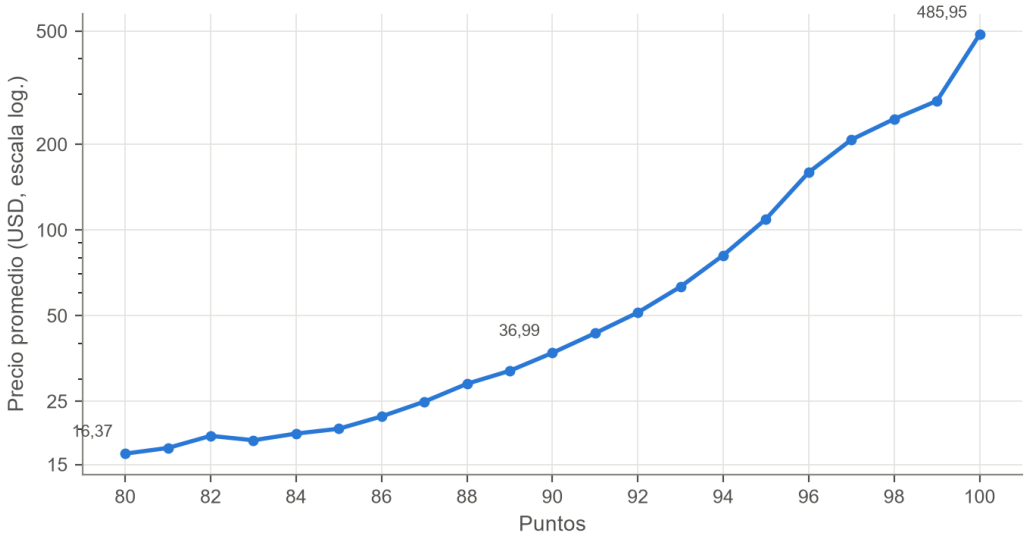


Figura 2. Precio promedio por puntuación (eje vertical en escala logarítmica).

El precio sube con cada punto pero no de forma lineal: de 80 a 90 puntos pasa de 16,37 a 36,99 dólares, y de 90 a 100 se multiplica por trece hasta 485,95. Por eso usé escala logarítmica en el gráfico, donde la curva queda casi recta. Igual la dispersión es enorme: el vino más caro del dataset (un Château les Ormes Sorbet de 3.300 dólares) tiene 88 puntos, y con 87 hay vinos de 5 dólares.

4.4 Subconsulta correlacionada (C06)

Busca vinos argentinos que cuestan más del triple del promedio de su provincia. La subconsulta depende de v.id_provincia, así que se evalúa por cada fila.

```
SELECT v.titulo,
       p.nombre AS provincia,
       r.puntos,
       r.precio,
       ROUND((SELECT AVG(r2.precio)
              FROM resena r2 JOIN vino v2 ON v2.id_vino = r2.id_vino
              WHERE v2.id_provincia = v.id_provincia), 2) AS promedio_provincia
FROM resena r
JOIN vino v ON v.id_vino = r.id_vino
JOIN provincia p ON p.id_provincia = v.id_provincia
JOIN pais pa ON pa.id_pais = p.id_pais
WHERE pa.nombre = 'Argentina'
  AND r.precio > 3 * (SELECT AVG(r2.precio)
                      FROM resena r2 JOIN vino v2 ON v2.id_vino = r2.id_vino
                      WHERE v2.id_provincia = v.id_provincia)
ORDER BY r.precio DESC, r.id_resena DESC
LIMIT 10;
```

titulo	provincia	puntos	precio	promedio_provincia
Viña Cobos 2013 Volturno Cabernet Sauvignon (Perdriel)	Mendoza Province	92	230,00	25,10
Viña Cobos 2012 Volturno Marchiori Vineyard Cabernet Sauvignon (Perdriel)	Mendoza Province	91	230,00	25,10
Viña Cobos 2010 Cobos Marchiori Vineyard Malbec (Mendoza)	Mendoza Province	95	225,00	25,10
Viña Cobos 2011 Marchiori Vineyard Block C2 Malbec (Perdriel)	Mendoza Province	92	215,00	25,10
Viña Cobos 2012 Marchiori Vineyard Block C2 Malbec (Perdriel)	Mendoza Province	91	210,00	25,10
Viña Cobos 2013 Malbec (Perdriel)	Mendoza Province	89	210,00	25,10

Resultado de C06 (primeras 6 de 10 filas).

Los primeros puestos son todos de Viña Cobos, entre 210 y 230 dólares, contra un promedio provincial de 25,10. Son vinos de viñedos únicos como Marchiori, con buenos puntajes pero no tan superiores como el precio haría pensar.

4.5 Cuartiles de precio (C10)

Con NTILE(4) dividí las reseñas con precio en cuatro grupos iguales, para ver si los vinos caros puntúan mejor sin elegir cortes a mano.

```
WITH cuartiles AS (
    SELECT puntos, precio, NTILE(4) OVER (ORDER BY precio) AS cuartil
    FROM resena
    WHERE precio IS NOT NULL
)
SELECT cuartil,
       MIN(precio) AS desde,
       MAX(precio) AS hasta,
       COUNT(*) AS resenas,
       ROUND(AVG(puntos), 2) AS puntaje_promedio,
       ROUND(100 * AVG(puntos >= 90), 1) AS pct_90_o_mas
FROM cuartiles
GROUP BY cuartil
ORDER BY cuartil;
```

cuartil	desde	hasta	resenas	puntaje_promedio	pct_90_o_mas
1	4,00	17,00	27899	86,03	6,6
2	17,00	25,00	27898	87,70	24,7
3	25,00	42,00	27898	88,94	46,0
4	42,00	3300,00	27898	90,99	73,6

Resultado de C10 (4 filas).

El puntaje sube en cada cuartil: el más barato (hasta 17 dólares) promedia 86,03 y solo el 6,6% llega a 90, mientras que el más caro (desde 42) promedia 90,99 y el 73,6% pasa los 90. Casi cinco puntos de diferencia es mucho para una escala que en la práctica va de 80 a 100.

4.6 Otras consultas del script

El resto no lo incluyo completo, pero hay un par de resultados que vale la pena mencionar. C09 encadena dos CTE y usa ROW_NUMBER() OVER (PARTITION BY pais ...) para sacar las tres variedades principales de los seis países con más reseñas: Estados Unidos es sobre todo Pinot Noir, Francia cortes bordeleses y España Tempranillo, mientras que en Italia y Portugal lo que más hay es "Red Blend" y "Portuguese Red", categorías genéricas que mezclan vinos muy distintos. C05 arma un ranking de bodegas con al menos 30 reseñas (gana Cayuse con 94,07) y C07 busca calidad-precio (93 puntos o más por 20 dólares o menos; el primero es un Madiran francés de 12 dólares). C08 arma el perfil de los catadores: Roger Voss escribió 23.560 reseñas, y Anne Krebiehl promedia 90,63 contra 85,78 de Alexander Peartree, aunque eso no quiere decir que uno sea más exigente, porque cada uno reseña países distintos. Las demás usan NOT EXISTS (C11), LAG() para la evolución por cosecha (C12), una subconsulta escalar en el HAVING (C13) y RANK() para la mejor bodega de cada país (C14).

5. Análisis multidimensional

Para el análisis tomé resena como tabla de hechos (medidas: cantidad, puntos y precio) y como dimensiones país, provincia, región, variedad, rango de precio y categoría de puntaje. El modelo ya tenía esa forma, en un esquema copo de nieve por la jerarquía geográfica. Para no repetir ocho JOIN en cada consulta usé la vista v_hechos_resena. MySQL no tiene motor OLAP, pero todas las operaciones se pueden escribir en SQL (sección 7 del script, M01 a M07).

5.1 Roll-up: país y rango de precio (M01)

Uso GROUP BY ... WITH ROLLUP y GROUPING() para poner un rótulo en las filas de subtotal en vez del NULL.

```
SELECT IF(GROUPING(pais) = 1, '(TOTAL)', pais) AS pais,
       IF(GROUPING(rango_precio) = 1, '(subtotal)', rango_precio) AS rango_precio,
       COUNT(*) AS resenas,
       ROUND(AVG(puntos), 2) AS puntaje_promedio
FROM v_hechos_resena
WHERE pais IN ('Argentina', 'Chile', 'Spain')
GROUP BY pais, rango_precio WITH ROLLUP;
```

pais	rango_precio	resenas	puntaje_promedio
Argentina	1. < 15	1278	84,62
Argentina	2. 15-29	1500	86,66
Argentina	3. 30-59	474	89,50
Argentina	4. 60-99	154	91,29
Argentina	5. >= 100	95	91,95
Argentina	Sin precio	43	86,72
Argentina	(subtotal)	3544	86,65
Chile	1. < 15	1982	84,97
Chile	2. 15-29	1575	87,13
Chile	3. 30-59	381	89,13
Chile	4. 60-99	150	90,99
Chile	5. >= 100	42	91,29
Chile	Sin precio	54	86,31
Chile	(subtotal)	4184	86,46
Spain	1. < 15	1943	85,07
Spain	2. 15-29	2553	87,16
Spain	3. 30-59	1039	89,45
Spain	4. 60-99	301	90,87
Spain	5. >= 100	213	92,03
Spain	Sin precio	67	86,97
Spain	(subtotal)	6116	87,24
(TOTAL)	(subtotal)	13844	86,85

Resultado de M01 (22 filas).

En los tres países el puntaje sube con el precio de forma muy parecida, de unos 85 puntos por debajo de 15 dólares a unos 92 por encima de 100. Lo que cambia es cómo se reparten los vinos entre rangos: España tiene más vinos caros que Chile y eso explica buena parte de que su promedio sea más alto (87,24 contra 86,46). Dentro de cada franja los tres están a menos de un punto.

5.2 Pivot: país y categoría de puntaje (M03)

MySQL no tiene PIVOT, así que cada columna es una agregación condicional. AVG(categoria = 'Excelente') da directamente la proporción.

```
SELECT pais,
       COUNT(*) AS resenas,
       ROUND(100 * AVG(categoria = '1. Aceptable'), 1) AS pct_aceptable,
       ROUND(100 * AVG(categoria = '2. Bueno'), 1) AS pct_bueno,
       ROUND(100 * AVG(categoria = '3. Muy bueno'), 1) AS pct_muy_bueno,
       ROUND(100 * AVG(categoria = '4. Excelente'), 1) AS pct_excelente,
       ROUND(100 * AVG(categoria IN ('5. Soberbio', '6. Clásico')), 1) AS pct_soberbio_o_mas
FROM v_hechos_resena
WHERE pais IS NOT NULL
GROUP BY pais
HAVING COUNT(*) >= 1000
ORDER BY pct_excelente + pct_soberbio_o_mas DESC;
```

pais	resenas	pct_aceptable	pct_bueno	pct_muy_bueno	pct_excelente	pct_soberbio_o_mas
Austria	3034	0,2	8,1	31,2	51,3	9,1
Germany	1992	0,2	8,5	34,6	48,9	7,8
France	20353	0,8	23,6	33,3	35,1	7,2
US	50457	2,9	23,6	32,4	35,7	5,4
Australia	2197	1,7	23,3	34,6	36,1	4,2
Portugal	5256	1,0	31,1	31,9	31,5	4,5
New Zealand	1278	1,2	23,2	40,8	34,1	0,6
Italy	17940	0,5	20,8	44,2	29,9	4,6
South Africa	1301	0,4	29,6	38,6	30,1	1,4
Spain	6116	6,3	35,3	34,5	21,8	2,2
Argentina	3544	8,8	41,2	29,8	18,7	1,5
Chile	4184	6,8	45,1	33,5	14,4	0,2

Resultado de M03 (12 filas).

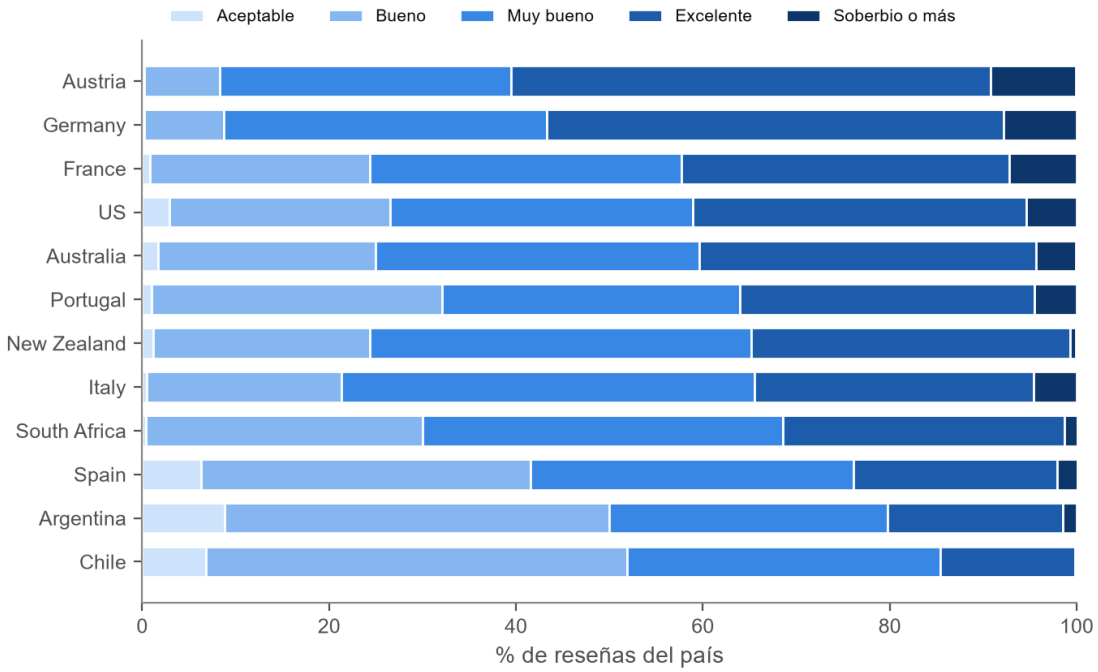


Figura 3. Distribución de categorías de puntaje por país (países con al menos 1.000 reseñas).

Austria y Alemania tienen más de la mitad de sus vinos en Excelente o más, mientras que en Chile y Argentina la categoría más común es Bueno (83-86). Todos tienen algún vino soberbio; la diferencia está en dónde se concentra la mayoría.

5.3 Dice: países, variedades y precio (M05)

Restringí tres dimensiones a la vez: tres países, dos variedades y dos rangos de precio.

```
SELECT pais,
       variedad,
```

```
       rango_precio,
       COUNT(*) AS resenas,
       ROUND(AVG(puntos), 2) AS puntaje_promedio
FROM v_hechos_resena
WHERE pais IN ('Argentina', 'Chile', 'US')
  AND variedad IN ('Malbec', 'Cabernet Sauvignon')
  AND rango_precio IN ('2. 15-29', '3. 30-59')
GROUP BY pais, variedad, rango_precio
HAVING COUNT(*) >= 10
ORDER BY variedad, rango_precio, puntaje_promedio DESC;
```

pais	variedad	rango_precio	resenas	puntaje_promedio
Chile	Cabernet Sauvignon	2. 15-29	256	87,48
US	Cabernet Sauvignon	2. 15-29	1487	86,88
Argentina	Cabernet Sauvignon	2. 15-29	253	86,46
Chile	Cabernet Sauvignon	3. 30-59	54	89,56
Argentina	Cabernet Sauvignon	3. 30-59	37	89,46
US	Cabernet Sauvignon	3. 30-59	2457	89,00
Chile	Malbec	2. 15-29	62	87,39
Argentina	Malbec	2. 15-29	609	87,24
US	Malbec	2. 15-29	149	87,15
Argentina	Malbec	3. 30-59	231	89,94
US	Malbec	3. 30-59	282	88,90

Resultado de M05 (11 filas).

Con el precio fijo las conclusiones de C01 cambian. Un Cabernet Sauvignon chileno de 15 a 29 dólares (87,48) puntúa mejor que uno estadounidense del mismo rango (86,88), y entre 30 y 59 dólares el Malbec argentino (89,94) le gana al de Estados Unidos (88,90). Los promedios nacionales mezclaban calidad con cuántos vinos caros tiene cada país.

5.4 Matriz país y rango de precio (M07)

```
SELECT pais,
       ROUND(AVG(CASE WHEN rango_precio = '1. < 15' THEN puntos END), 2) AS menos_15,
       ROUND(AVG(CASE WHEN rango_precio = '2. 15-29' THEN puntos END), 2) AS de_15_a_29,
       ROUND(AVG(CASE WHEN rango_precio = '3. 30-59' THEN puntos END), 2) AS de_30_a_59,
       ROUND(AVG(CASE WHEN rango_precio = '4. 60-99' THEN puntos END), 2) AS de_60_a_99,
       ROUND(AVG(CASE WHEN rango_precio = '5. >= 100' THEN puntos END), 2) AS de_100_o_mas
FROM v_hechos_resena
WHERE pais IN (SELECT pais FROM (
                  SELECT pais FROM v_hechos_resena
                  WHERE pais IS NOT NULL
                  GROUP BY pais ORDER BY COUNT(*) DESC LIMIT 10) t)
GROUP BY pais
ORDER BY pais;
```

pais	menos_15	de_15_a_29	de_30_a_59	de_60_a_99	de_100_o_mas
Argentina	84,62	86,66	89,50	91,29	91,95
Australia	85,72	88,04	90,08	91,36	93,00
Austria	87,90	89,46	91,48	92,87	94,04
Chile	84,97	87,13	89,13	90,99	91,29
France	85,88	87,84	89,94	91,40	93,38
Germany	87,41	89,09	90,93	91,70	93,25
Italy	85,78	87,53	89,50	90,91	92,31
Portugal	86,09	88,66	90,91	92,31	94,08
Spain	85,07	87,16	89,45	90,87	92,03
US	85,60	87,37	89,42	91,21	92,56

Resultado de M07 (10 filas).

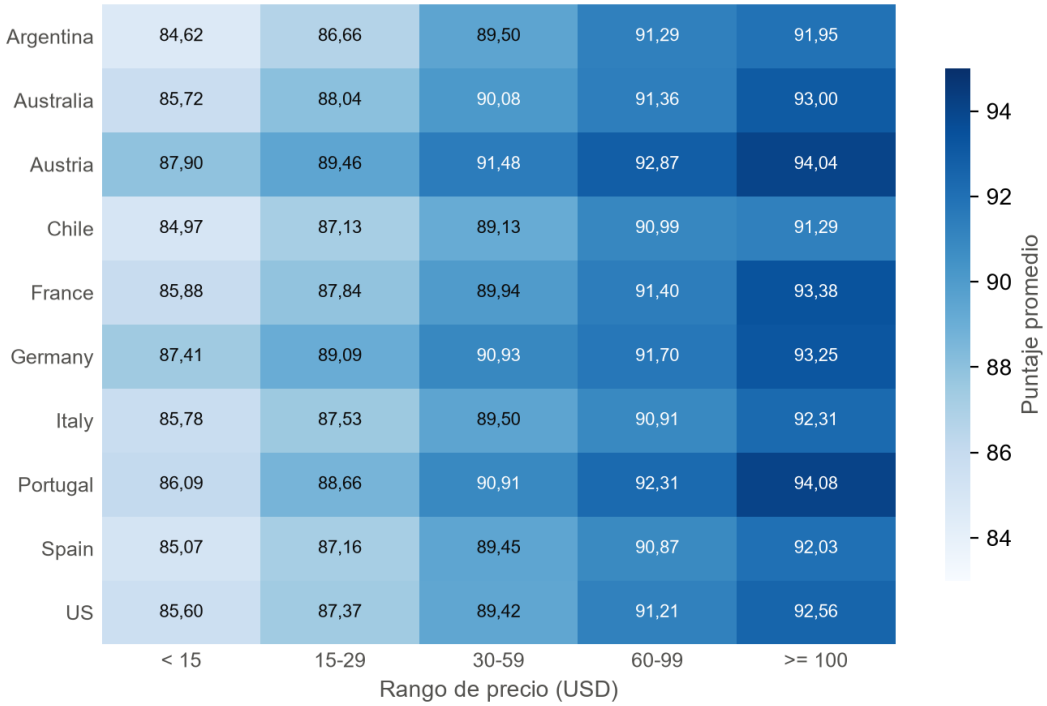


Figura 4. Puntaje promedio por país y rango de precio.

En todas las filas el puntaje crece de izquierda a derecha; lo que cambia es desde dónde arranca cada país. Austria y Alemania están arriba en casi todas las columnas, al punto de que un austríaco de menos de 15 dólares (87,90) supera en promedio a los argentinos, chilenos, españoles, estadounidenses, italianos y franceses de 15 a 29. En el rango más caro, Portugal (94,08) y Austria (94,04) son los únicos arriba de 94.

5.5 Drill-down, slice y cubo

Las otras tres operaciones están en el script. El drill-down en Argentina (M02) muestra que el 85,7% de las reseñas son de Mendoza, con mucha diferencia entre regiones (Perdriel 90,28, Maipú 84,96), y además dejó ver un problema que no había detectado: "Uco Valley" y "Valle de Uco" figuran como regiones distintas. El slice de Malbec (M04) da que Argentina tiene el 57% de las reseñas de esa variedad (1.402 de 2.446) pero el francés, casi todo de Cahors, puntúa más (89,65 contra 87,46). El cubo país por variedad (M06) lo armé con UNION ALL de los cuatro agrupamientos porque MySQL no tiene GROUP BY CUBE.

6. Desafíos

El primer problema apareció antes de escribir SQL: el archivo no era el CSV original sino una versión rota por Excel. Si lo importaba así iba a cargar texto ilegible y unas 430 reseñas con columnas corridas, así que tuve que limpiarlo antes de poder cargar nada.

En el diseño, lo más difícil fue la granularidad de vino. La idea de un vino por título con su precio no cerraba con los datos (el mismo título tenía precios distintos), y lo resolví pasando el precio a resena y armando la clave natural con una columna generada. Algo parecido con provincia y región: con FK simples se colaban combinaciones imposibles, y tuve que pasar a una FK compuesta.

La carga también falló varias veces antes de salir bien. Por ejemplo, al principio había puesto usuario_twitter como UNIQUE porque parecía lógico, y la carga se cortó porque dos catadores comparten la misma cuenta. Lo mismo con los duplicados: hasta que no los saqué, todos los conteos y promedios daban inflados.

Por último, la consulta C06 tardaba muchísimo cuando corría el script completo (casi tres minutos), mientras que sola tardaba unos 20 segundos. Investigando vi que MySQL no tenía actualizadas las estadísticas de las tablas recién cargadas, y con un ANALYZE TABLE después de la carga se solucionó. Igual sigue siendo la consulta más pesada, por eso la limité a un solo país.

7. Conclusiones y trabajo futuro

Pasar del archivo plano a nueve tablas normalizadas eliminó la repetición del CSV y dejó en manos del SGBD reglas que antes nadie controlaba: puntajes en escala, precios positivos, regiones coherentes con su provincia y reseñas siempre asociadas a un vino. Las pruebas de integridad y la transacción revertida muestran que funcionan.

Sobre los vinos, el precio y el puntaje están asociados de forma consistente (casi cinco puntos entre el cuartil más barato y el más caro) en todos los países y rangos que miré. Lo que más me sorprendió es lo engañosos que son los promedios por país si no se controla el precio: Chile y Argentina parecen atrasados en C01, pero dentro de la misma franja compiten de igual a igual con Estados Unidos. Austria, Alemania y Portugal se destacan en vinos baratos. Hay que tener en cuenta los límites del dataset: sobrerrepresenta a Estados Unidos, el puntaje depende en parte del catador (y cada uno cubre regiones distintas) y categorías como "Red Blend" mezclan vinos que no tienen nada que ver.

Lo que quedó sin aprovechar es la columna description, con casi 120.000 notas de cata. Con procesamiento de lenguaje natural se podría intentar predecir la variedad a partir del texto, que es justamente lo que plantea el enunciado con los sommeliers, y los índices FULLTEXT de MySQL servirían para una primera exploración. También estaría bueno armar una tabla de sinónimos para casos como "Uco Valley" y "Valle de Uco", y probar un modelo de regresión del puntaje que separe el efecto del catador.

Referencias

Oracle Corporation. (2024). MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/

Thoutt, Z. (2017). Wine Reviews [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/zynicide/wine-reviews